Pautas del Proyecto Final: Aplicación Web Full Stack

1. Definición del Tema y Alcance

- **Tema Libre:** Cada grupo debe elegir una problemática real que pueda solucionarse mediante una aplicación web (ej. un e-commerce, una red social, un sistema de gestión hospitalaria, etc.).
- **Requisito de Complejidad:** La aplicación no puede ser una simple página informativa. Debe ser una **Aplicación Web Dinámica** que gestione datos, usuarios y lógica de negocio.

2. Requisitos Técnicos Obligatorios (El "Core")

Para cumplir con el contenido programático modernizado, el proyecto debe integrar:

A. Arquitectura y Frontend

- **Framework Moderno:** Uso obligatorio de React.js, Vue.js o Next.js.
- **Interfaz de Usuario (UI):** Debe ser **Totalmente Responsive** (adaptable a móviles, tablets y desktop) utilizando CSS moderno (Tailwind CSS o CSS Modules).
- **Consumo de API:** El frontend debe comunicarse con el backend de forma asíncrona (usando fetch, axios o TanStack Query).
- **Gestión de Estado:** Implementación de estado global para datos del usuario o carrito/preferencias (Zustand, Redux o Context API).

B. Backend y API

- **Servidor Propio:** Desarrollado en Node.js (Express/NestJS) o Python (FastAPI/Django). No se permite el uso exclusivo de Firebase (para evaluar la capacidad de programar lógica de servidor).
- **Diseño RESTful:** La API debe seguir las convenciones de nombres y verbos HTTP (GET, POST, PUT, DELETE).
- **Validación:** Todas las entradas de datos del cliente deben ser validadas en el servidor (ej. usando Zod o Joi).

C. Persistencia de Datos (Base de Datos)

- **Modelo de Datos:** Uso de una base de datos relacional (PostgreSQL/MySQL) o no relacional (MongoDB).
- **Operaciones CRUD:** La aplicación debe realizar al menos un ciclo completo de Crear, Leer, Actualizar y Eliminar registros complejos.

D. Seguridad

- **Autenticación:** Sistema de Login/Registro funcional.
- **Autorización:** Uso de **JSON Web Tokens (JWT)** para proteger rutas privadas.
- **Protección de Datos:** Las contraseñas en la base de datos **nunca** deben estar en texto plano (uso de hashing con Bcrypt o Argon2).

---

3. Entregables del Proyecto

El proyecto se evaluará en tres fases para asegurar el progreso:

1. **Propuesta:** Documento corto con el título, problemática, stack tecnológico elegido y un diagrama sencillo de la base de datos.
2. **Repositorio de Código (GitHub):**
   - Uso correcto de ramas (Gitflow).
   - Un archivo README.md profesional que explique cómo instalar y ejecutar el proyecto localmente.
3. **Despliegue (Demo en vivo):** La aplicación debe estar publicada en el servidor previsto para ello en clase, para que pueda ser probada por el docente.

---

4. Criterios de Evaluación (Rúbrica resumida)

Criterio	Descripción	Puntaje
Funcionalidad	¿La aplicación resuelve el problema planteado sin errores críticos?	30%
Arquitectura	Calidad del código, separación de carpetas y buen uso del stack.	20%
UI/UX	Diseño estético, usable y responsive.	20%
Seguridad	Manejo de sesiones, protección de rutas y hashing de claves.	15%
Despliegue	La app es accesible vía URL pública y está en un repositorio.	15%

---

5. Prohibiciones (Limitaciones Técnicas)

- No se permite el uso de plantillas de administración donde el código no sea autoría de los estudiantes.
- No se permite el uso de librerías que resuelvan todo el backend automáticamente (ej. No usar WordPress o similares).
- No se aceptan proyectos sin control de versiones (Git).